

ZERO DEPENDENCY HACKATHON · TRACK F · OPEN / WILDCARD

# llamini.cpp

A from-scratch llama.cpp core. Zero dependencies.

No ggml. No Python. No packages. libc and POSIX — and a Makefile.

c99

empty manifest

4 architectures

make → ./llamini

# The inference engine nobody looks at

llama.cpp is load-bearing infrastructure. If it were a black box, so would every local LLM tool on a laptop.

## Ollama

179.8k GitHub stars

100M+ Docker pulls

depends on llama.cpp directly — pulled in and patched at build time

## GPT4All · LM Studio

consumer apps

ship it under the hood

LM Studio's own blog credits "our llama.cpp engine"

## llama-cpp-python

10.6k stars

direct binding

the Python the rest of the stack actually calls

### The thought experiment, grounded.

CVE-2024-3094 (xz-utils) was planted by a contributor persona over two years. Caught by one engineer noticing odd SSH timing — not a scan. If that class of compromise landed in llama.cpp instead of sshd, it would ride into every tool above. This project does not replace that stack. It makes the core readable, auditable, and reproducible by one person in a weekend.

# The core, written out by hand

## GGUF parser

header, all 13 metadata types, tensor-info table, bounds-checked mmap

## 9 quant formats

F32 F16 Q4\_0 Q4\_1 Q5\_0 Q8\_0 Q4\_K Q5\_K Q6\_K

## Transformer pass

GQA/MQA attention, RoPE NORM+NEOX, SwiGLU/GeGLU, LayerNorm

## Two tokenizers

SentencePiece-BPE and byte-level BPE, from the file's own vocab

## Four families

LLaMA, Qwen2, GPT-2, Gemma — auto-detected from general.architecture

## What you do not install

- ggml / llama.cpp as a library
- sentencepiece / tiktoken
- numpy / a BLAS
- llama-cpp-python
- CMake, vcpkg, Conan
- any Python runtime

One command builds it.  
`make → ./llamini`

 Gordon

 Containers

 Images

 Volumes

 Kubernetes

 Builds

 Docker Hub

 Models

 MCP Toolkit **BETA**

 Logs

 Extensions  
Extensions are disabled.

Upgrade plan

[Containers](#) / llamini-demo



**llamini-demo**

 5306348343af 

 [llamini-demo-base:latest](#)

**STATUS**

Running (11 seconds ago)



**Logs**

Inspect

Bind mounts

Exec

Files

Stats



View and search logs for all containers in one place with the new Logs view.

View logs



Starting in 10s...

Starting in 9s...

Starting in 8s...

Starting in 7s...

Starting in 6s...

Starting in 5s...

Starting in 4s...

Starting in 3s...

Starting in 2s...

Starting in 1s...

=====

SCENE 2 -- git clone + make

=====

\$ git clone <https://github.com/hatif03/llamini.cpp.git> 



 Engine running



RAM 1.22 GB CPU 0.00% Disk: 5.78 GB used (limit 1006.85 GB)

 [Update version](#)



Q Search



ENG  
IN

08:20  
31-08-2026

# Six things that tape just proved

01

## One command

make. No CMake, no vcpkg, no package manager — gcc and a Makefile.

02

## Zero downloads

./llamini --test builds a synthetic model in memory. The binary works before any .gguf exists.

03

## Honest setup

--setup prints the curl command. HuggingFace is HTTPS-only; TLS or shelling out would break the rule.

04

## Real inference

GPT-2, real GGUF weights, byte-level BPE, a real transformer forward pass — live, at real speed.

05

## A checkable number

Teacher-forced perplexity 20.58 on GPT-2. Three orders of magnitude below a broken model.

06

## Empty manifest

Idd: libc and libm only. grep for every package file: no output.



# The proof, not the claim

```
$ ldd llamini
```

```
linux-vdso.so.1
```

```
libm.so.6 => /usr/lib/x86_64-linux-gnu/libm.so.6
```

```
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6
```

```
/lib64/ld-linux-x86-64.so.2
```

```
# libc and libm. Both standard. Nothing else.
```

```
$ grep -RIIE 'package.json|Cargo.toml|vcpkg.json|CMakeLists.txt' .
```

```
(no output = no matches)
```

## Linked libraries

# 2

libc + libm

vdso and ld-linux are the kernel and the dynamic linker, not packages.

## Why --setup prints curl

HuggingFace is HTTPS-only. Hand-rolling TLS, or quietly shelling out to curl, both break the rule. So the command is printed — you read it before you run it.

# What this replaces

| People actually install            | Replaced here by                                 |
|------------------------------------|--------------------------------------------------|
| ggml / llama.cpp as a library      | gguf.c + model.c + gpt2.c                        |
| sentencepiece / libsentencepiece   | tokenizer.c (SPM path, from the GGUF vocab)      |
| byte-level BPE (tiktoken-adjacent) | tokenizer.c (second path, rank-based merges)     |
| numpy / a BLAS                     | linear() in model.c (thread pool + lazy dequant) |
| llama-cpp-python, ctransformers    | the CLI is the binary                            |
| huggingface_hub / silent curl      | printed curl command + POSIX stat()              |
| CTest / Unity / Check              | --test + printf (C has no stdlib test runner)    |

# Four families, auto-detected from the file

| Model                 | Arch  | Params | Tokenizer      | PPL*  | Facts |
|-----------------------|-------|--------|----------------|-------|-------|
| TinyLlama-1.1B-Chat   | llama | 1.1B   | SentencePiece  | 13.28 | 1 / 3 |
| Qwen2.5-0.5B-Instruct | qwen2 | 0.5B   | byte-level BPE | 10.84 | 3 / 3 |
| GPT-2                 | gpt2  | 124M   | byte-level BPE | 20.58 | 0 / 3 |
| Gemma-2b-it           | gemma | 2.5B   | SentencePiece  | 73.05 | 2 / 3 |

\* Teacher-forced perplexity over the same ~35-token English corpus. Not comparable across vocab sizes — compare each number to its own random-guess ceiling: TinyLlama ~32,000 · Qwen2.5 ~151,936 · GPT-2 ~50,257 · Gemma ~256,128. All four sit two-to-three orders of magnitude below chance. GPT-2 is its own module (fused QKV, LayerNorm, learned positions, no RoPE). Qwen2 needed NEOX RoPE, QKV bias, and Q5\_0 — found by probing the real file, not assumed from the “Q4\_K\_M” name. Gemma was untestable at ~10–12 GB until lazy dequant dropped it to ~1.5 GB.



# Not “it looks like English”

**20.58**

**GPT-2 PPL**

ceiling ~50,257

**13.28**

**TinyLlama PPL**

ceiling ~32,000

**10.84**

**Qwen2.5 PPL**

ceiling ~151,936

**73.05**

**Gemma-2b PPL**

ceiling ~256,128

## How the number is made

Teacher forcing: at every position, measure the probability the real logits assign to the actual next token. Never sampling. A uniform-random guess over a vocab has perplexity equal to the vocab size. 20.58 vs ~50,257 is the claim that the forward pass is doing language modeling, not emitting token-shaped noise.

No reference llama.cpp was linked to diff against — staying zero-dep. So this is a self-contained check, plus known-fact completions (Qwen 3/3, Gemma 2/3, TinyLlama 1/3, GPT-2 0/3). A 124M model missing “Paris” is a capability limit, not papered over as a pass.

# Less memory. Same machine. Unmodified upstream.

| Model          | llamini RSS | llama.cpp RSS | Winner                |
|----------------|-------------|---------------|-----------------------|
| TinyLlama-1.1B | ~710 MB     | ~1.12 GB      | llamini               |
| Qwen2.5-0.5B   | ~480 MB     | ~558 MB       | llamini               |
| Gemma-2b       | ~1.54 GB    | ~2.66 GB      | llamini               |
| GPT-2-124M     | ~831 MB     | ~169 MB       | llama.cpp (disclosed) |

Resident memory dropped ~6–7× after the CPU pass (4.4–4.9 GB → 700 MB–1.5 GB) — weights dequantize on demand from the mmap’d file, not a persistent f32 copy. That is also how Gemma-2b went from untestable to ~1.54 GB resident.

Built unmodified ggml-org/llama.cpp (commit 0b5be7e) from source, outside this repo, never vendored, never invoked by the submitted binary. GPT-2 loses because gpt2.c was not brought onto the lazy path — written down in STDLIB.md, not a mystery.

Do not claim a speed win. Timing on this WSL2 guest is noisy for both engines (llama.cpp’s own ± often exceeds its mean).

# Two changes. Bit-identical output.

## 01 Persistent thread pool

linear() used to pthread\_create / pthread\_join on every call — about nine spawns per layer per token. TinyLlama's 22 layers: ~200 thread births to emit one word.

Now: a pool created once. Workers block on a condvar between jobs. No work-stealing, no NUMA — one hot call site.

Caught on paper before running: a naive broadcast lets idle slots read stale job data from the previous dispatch. Each worker now checks that its slot actually got work this round.

## 02 On-demand row-chunk dequant

CPU batch-1 inference is memory-bandwidth bound. ggml never materializes a full f32 tensor — that's the actual lever, not FLOPs.

Big weights (embeddings, lm\_head, every projection) dequantize in 256-row chunks straight out of the mmap'd file, into a small scratch buffer, then the existing f32 dot product.

Chunk size is fixed, not "split by core count." Gemma's 256,128-row embed would otherwise need a multi-hundred-MB buffer per worker — the problem, moved down one level.

Measured: ~4.4–4.9 GB → ~700 MB–1.5 GB (6–7×). Gemma-2b went from untestable to verified at ~1.5 GB. Acceptance bar was bit-identical generated text against the pre-change binary, not "doesn't crash."

Still simpler than ggml, on purpose: no int8×int8 kernels, lazy path is serial, GPT-2 weights were not brought onto it. That's the RSS loss on slide 10.

# One concern per file

## gguf.c

mmap the file. Parse header, metadata, tensor-info. Don't touch weights yet.

## main.c

Read general.architecture. gpt2 → gpt2.c. Else LLaMA-family (llama / qwen2 / gemma).

## model.c

Config from namespaced keys. Big weights stay lazy. Norms/biases stay eager.

## tokenizer.c

Vocab lives in the GGUF. SPM or byte-level BPE, dispatched at load time.

## generate.c

Prefill every prompt token into the KV cache, then sample. Same forward\_step as --bench.

## kv\_cache.c

One cache per layer, sized to kv\_dim — not the full embedding width.

# What this is not

## Single-turn only

Every chat line resets every layer's KV cache. No conversational memory.

## Chat template is TinyLlama-only

Best-effort from tokenizer\_config.json, not verified against transformers. Qwen2 (ChatML) and GPT-2 are raw-completion.

## Partial quant coverage

The 9 formats the four files actually use. Q2\_K / Q3\_K / Q8\_K / IQ\* return -1, never silently misread.

## Sampling is small

Greedy default. --temp adds temperature + top-p. No top-k, no repetition penalty, no --seed.

## GPT-2 weights still eager

That's why GPT-2 RSS loses to llama.cpp. Written down, not left to be discovered in the table.

## Lazy path is simpler than ggml

Dequant to f32 scratch, then dot. No int8×int8 kernels. Serial, not thread-pooled — chunk size is for bounded memory, not peak FLOPs.

## No GPU, no LoRA, no batching

CPU, batch-1, one process. A weekend core, not a production engine.

## Mistral / Gemma2 / BERT skipped

Mistral's GGUF arch is literally "llama". Gemma2 adds softcapping and sliding windows. BERT is a different product shape.



NOTHING HERE IS A BLACK BOX

# Every package. Every file. Every mistake.

## STDLIB.md

Every package this would normally need, and exactly what replaced it.

## ARCHITECTURE.md

Every file, in order, for a human who has never seen the tree.

## BUILDLOG.md

What was tried, including what made things worse.

## deps-proof.txt

Idd, the empty grep, the machine the numbers came from.

```
make && ./llamini --test
```

Questions — start with the tape, STDLIB.md, or the memory table.